UNIVERSIDADE DE FORTALEZA
CENTRO DE CIÊNCIAS TECNOLÓGICAS
CURSO: CIÊNCIA DA COMPUTAÇÃO

ANÁLISE DE DESEMPENHO DE ALGORITMOS DE ORDENAÇÃO EM AMBIENTES SEQUENCIAIS E PARALELOS EM JAVA

Autor: Vitor Vargas Cardoso.

Professor: Izequiel. Turma: T296-26. Disciplina: Computação Paralela e Concorrente.

Palavras-chave: Ordenação. Paralelismo. Java. Desempenho. ForkJoinPool.

Resumo

Este trabalho compara implementações sequenciais e paralelas de Quick Sort, Merge Sort, Insertion Sort e Selection Sort em Java. Foram utilizadas entradas aleatórias, crescentes e decrescentes, com 1.000, 5.000 e 10.000 elementos, execução sequencial e pools com paralelismo de duas, quatro e oito threads. Cinco amostras por configuração totalizaram 720 medições. As versões paralelas de Insertion e Selection adotaram ordenação de blocos seguida de fusão. Para 10.000 elementos aleatórios, as razões entre as médias sequencial e paralela com oito threads foram 1,48 para Quick e 2,08 para Merge. Os ganhos elevados dos métodos híbridos também refletem redução de trabalho, e não apenas concorrência. A ampliação do paralelismo não melhorou todos os cenários. Conclui-se que distribuição da entrada, estratégia algorítmica e custo de coordenação precisam ser considerados em conjunto, respeitando a dispersão e os limites do experimento.

Introdução

A ordenação reorganiza elementos segundo uma relação de ordem. Sua eficiência depende da estratégia adotada e das características dos dados. Neste estudo, comparamos quatro métodos aplicados a vetores de inteiros, buscando observar como a execução concorrente altera o tempo em diferentes tamanhos e distribuições de entrada.

Insertion Sort insere cada elemento no prefixo já ordenado; Selection Sort seleciona sucessivamente o menor elemento restante. Ambos podem apresentar custo quadrático, enquanto Insertion aproveita entradas já ordenadas com percurso linear (Sedgewick; Wayne, s.d.a). Merge Sort divide e combina subproblemas por fusão; Quick Sort particiona os elementos em torno de um pivô e ordena as regiões resultantes (Sedgewick; Wayne, s.d.b; s.d.c).

O objetivo é comparar os tempos e a dispersão das versões implementadas, registrando os resultados em CSV e variando o paralelismo configurado. Os métodos exigidos são de

ordenação, embora o enunciado empregue o termo busca. O estudo distingue paralelização recursiva de adaptações híbridas para evitar atribuir toda redução de tempo ao uso de threads.

Metodologia

O projeto foi implementado para Java 21 e compilado com JDK 24. A execução analisada ocorreu em 30 de outubro de 2026, em macOS com arquitetura aarch64 e dez processadores lógicos disponíveis para a JVM. O registro automático não inclui modelo da CPU, quantidade de memória ou carga do sistema; essa ausência limita a caracterização do ambiente.

Quick Sort utiliza partição em três vias, pivô obtido na posição central e recursão sequencial no lado menor para limitar a pilha. Sua versão paralela distribui regiões disjuntas. Merge Sort ordena metades e realiza fusão sequencial. As tarefas utilizam RecursiveAction e ForkJoinPool dedicado, mecanismo que admite paralelismo configurável e distribuição dinâmica de tarefas (Oracle, 2025).

Insertion e Selection paralelos dividem o vetor em blocos, executam o método correspondente em cada bloco e combinam os resultados por fusão. Essas versões são híbridas: a divisão reduz o trabalho quadrático nos blocos e altera a estratégia em relação ao serial. A criação de subtarefas encerra quando o orçamento de tarefas chega a um ou o trecho contém até 256 elementos. Configurar threads no pool não equivale a fixar núcleos físicos ou assegurar ocupação simultânea.

Foram combinados quatro algoritmos, três tamanhos (1.000, 5.000 e 10.000), três distribuições (aleatória, crescente e decrescente) e quatro configurações (serial, duas, quatro e oito threads), com cinco amostras: 4 × 3 × 3 × 4 × 5 = 720 medições. Os vetores aleatórios usam inteiros uniformes no intervalo [0, 10n); as entradas crescentes e decrescentes derivam da ordenação desses mesmos valores. Tamanho e intervalo variam juntos, sem isolamento dos dois fatores.

Cada configuração recebe três execuções de aquecimento antes das amostras. As sementes das cinco amostras são 2027 a 2031, mantendo dados iguais entre métodos e configurações dentro de cada amostra. A ordem das configurações é embaralhada de forma reproduzível. Os pools são reutilizados; a geração, a clonagem do vetor, a validação e a escrita do arquivo ficam fora da janela medida por System.nanoTime. Despacho de tarefas, alocações internas e fusões integram o tempo.

Todas as saídas foram comparadas integralmente com uma cópia ordenada por Arrays.sort fora da medição; o conjunto experimental contém 720 saídas válidas. Uma verificação adicional aprovou 800 casos envolvendo vetores vazios, pequenos, repetidos e ordenados. O CSV registra algoritmo, modo, tamanho, distribuição, limite de valores, threads, amostra, semente, tempos em nanossegundos e milissegundos e validade.

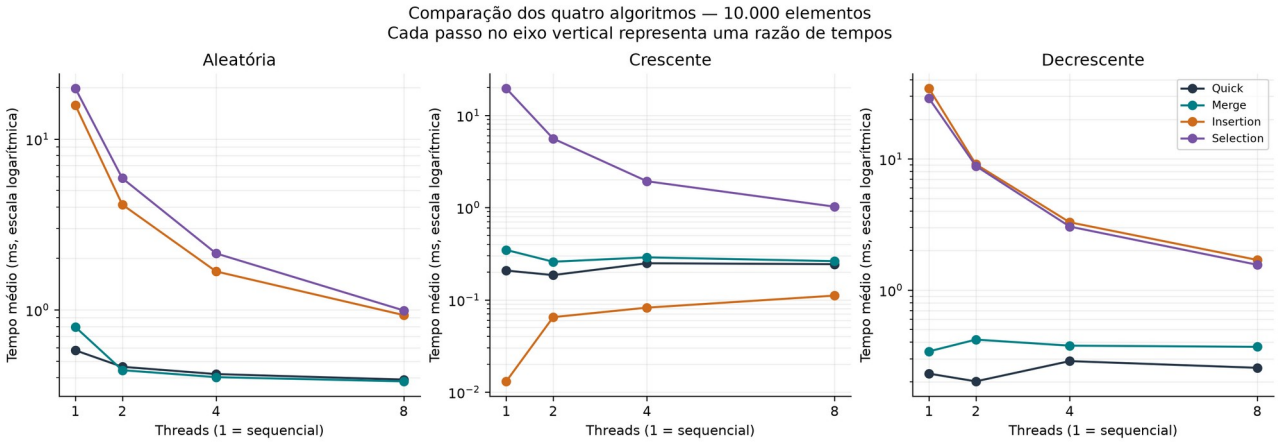
A análise é descritiva, com média, mediana e desvio padrão amostral. Definimos speedup como S(p) = média do tempo sequencial / média do tempo da configuração p; eficiência como E(p) = S(p)/p. Para os métodos híbridos, essas razões comparam estratégias com trabalho diferente e

não representam aproveitamento isolado do hardware. As barras dos gráficos de tempo mostram um desvio padrão, não intervalos de confiança.

Resultados e Discussão

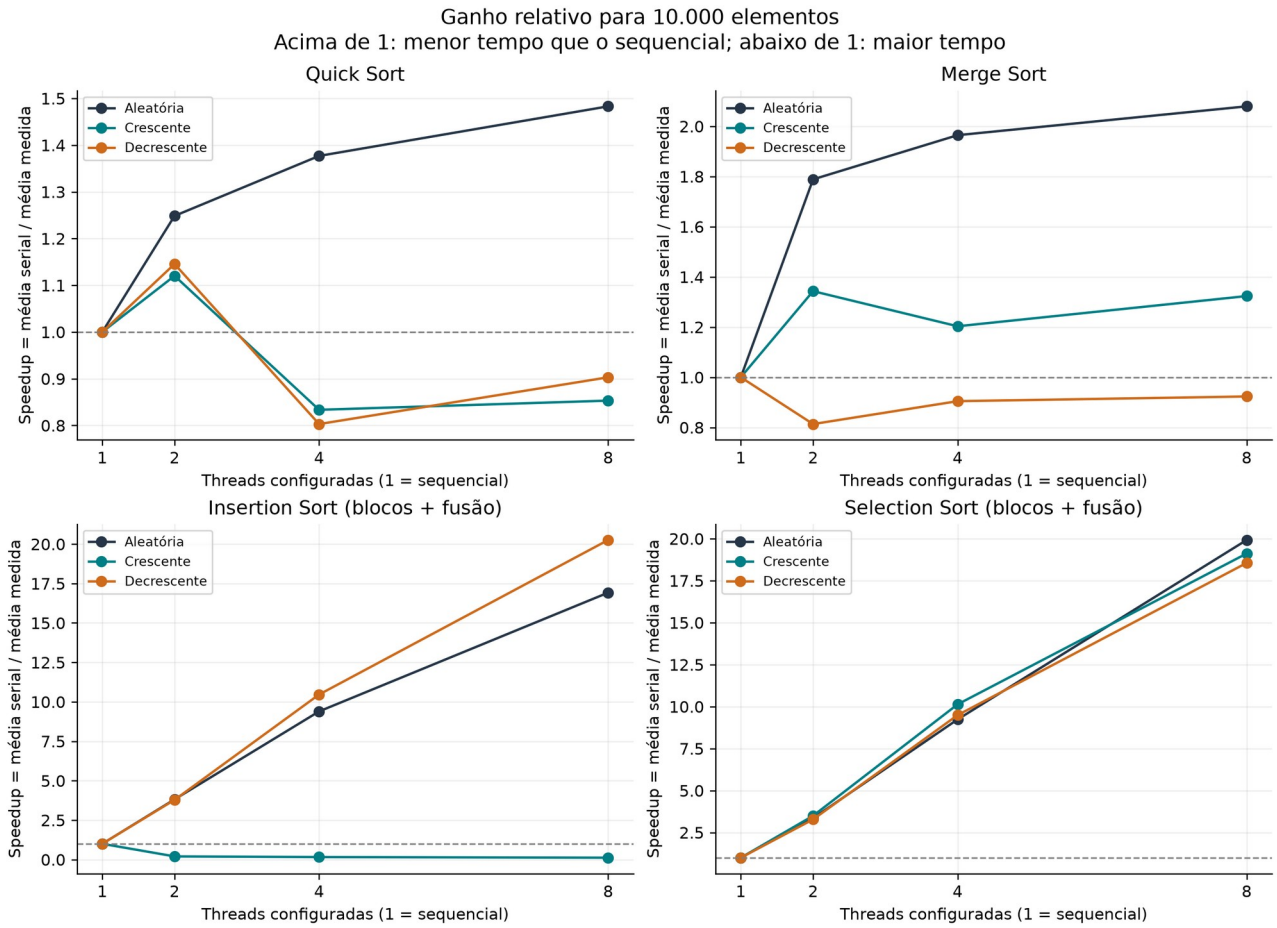
Para 10.000 elementos aleatórios, Quick Sort apresentou média de 0,5798 ms no modo sequencial e 0,3909 ms com oito threads, resultando em speedup de 1,48. Merge Sort apresentou 0,7937 ms e 0,3817 ms, respectivamente, com speedup de 2,08. Os valores paralelos são próximos; sua pequena diferença não estabelece superioridade geral sem repetições adicionais e avaliação da dispersão. A Figura 1 compara os tempos absolutos nas três distribuições.

Figura 1 – Tempos médios para 10.000 elementos em escala logarítmica. Fonte: dados do experimento (2026).



O aumento do paralelismo não produziu aceleração monotônica. Para Quick Sort crescente, duas threads tiveram média de 0,1852 ms, ante 0,2074 ms no serial e 0,2432 ms com oito threads. Para Merge Sort decrescente, a média serial de 0,3414 ms ficou abaixo de todas as médias paralelas. A Figura 2 evidencia ganhos e perdas em relação à referência sequencial.

Figura 2 – Speedup para 10.000 elementos. Os painéis usam escalas verticais próprias. Fonte: dados do experimento (2026).

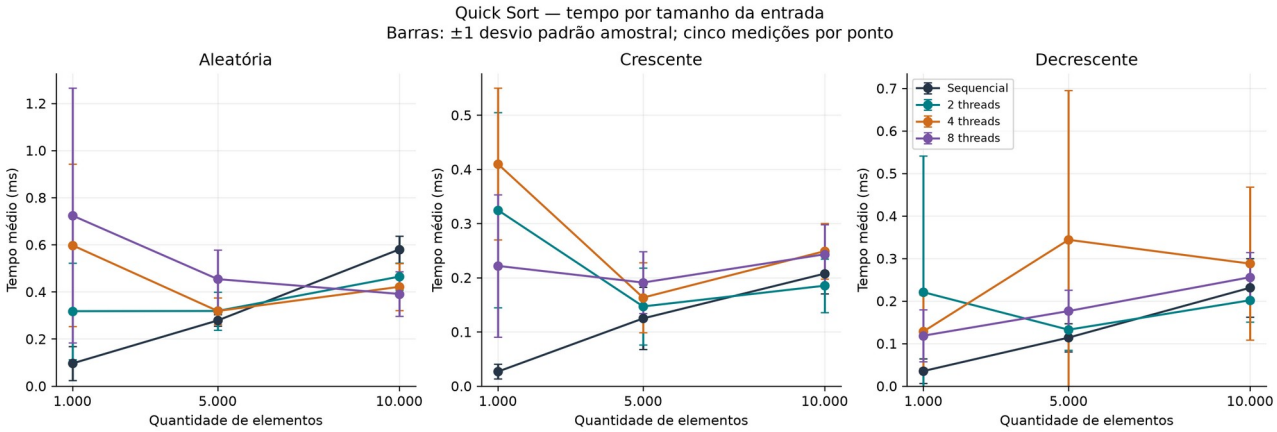


Insertion Sort foi especialmente sensível à entrada. Com 10.000 elementos crescentes, a média serial foi 0,0131 ms, enquanto a versão híbrida com oito threads apresentou 0,1110 ms, cerca de 8,47 vezes mais tempo. O serial evita deslocamentos nessa entrada, ao passo que o híbrido mantém tarefas e fusões. Na entrada decrescente, os tempos correspondentes foram 34,3838 ms e 1,6973 ms, com speedup de 20,26.

Selection Sort aleatório passou de 19,7679 ms no serial para 0,9923 ms com oito threads, ganho de 19,92. Ganhos superiores ao paralelismo configurado decorrem também da menor quantidade de trabalho nos blocos. A soma de custos quadráticos de p blocos equilibrados é proporcional a n^2/p, antes de considerar simultaneidade e fusões. Portanto, tais razões não demonstram eficiência física superior a 100%.

Os ganhos relativos também não determinam o menor tempo absoluto: no cenário aleatório com oito threads, Insertion híbrido apresentou 0,9320 ms e Selection híbrido 0,9923 ms, acima de Quick e Merge. A Figura 3 mostra como os tempos de Quick variaram com tamanho e distribuição; os demais gráficos completos acompanham o repositório.

Figura 3 – Quick Sort por tamanho e distribuição; média ± um desvio padrão. Fonte: dados do experimento (2026).



A dispersão foi elevada em algumas configurações. Em Insertion crescente com 1.000 elementos e oito threads, a média foi 0,7520 ms, a mediana 0,1280 ms e o desvio padrão 1,4250 ms. O coeficiente de variação, razão entre desvio padrão e média, foi 189,5%. Esse resultado mostra que a média isolada pode representar mal amostras assimétricas. Nenhuma observação foi excluída.

Tempos inferiores a um milissegundo e apenas cinco amostras tornam diferenças pequenas sensíveis às condições de execução. Sem instrumentação específica, não é possível atribuir observações isoladas à compilação JIT, coleta de lixo ou agendamento. O estudo utiliza uma única JVM, três aquecimentos e não aplica testes inferenciais. Para aprofundamento, propomos execuções em processos independentes, maior aquecimento e o uso de um harness específico para benchmarks da JVM, como JMH (OpenJDK, s.d.).

Conclusão

Os resultados mostram que o benefício do paralelismo depende do algoritmo, da distribuição e do tamanho da entrada. Na entrada aleatória com 10.000 elementos, oito threads reduziram as médias de Quick e Merge, enquanto entradas já ordenadas favoreceram configurações com menos threads e, para Insertion, a execução sequencial.

As adaptações de Insertion e Selection alcançaram ganhos relativos expressivos, mas alteraram o trabalho computacional por meio de blocos e fusões. Assim, o experimento demonstra o desempenho dessas estratégias completas, sem isolar o efeito das threads. A dispersão elevada em parte das configurações e a caracterização parcial do ambiente restringem a generalização. Novos experimentos devem ampliar tamanhos, repetições e processos JVM e comparar uma mesma estratégia híbrida também com um único trabalhador.

Referências

OPENJDK. Java Microbenchmark Harness (JMH). [S. l.], [s. d.]. Disponível em: https://github.com/openjdk/jmh. Acesso em: 30 out. 2026.

ORACLE. ForkJoinPool: Java Platform, Standard Edition 24 API Specification. [S. l.], 2025. Disponível em: https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/util/concurrent/ForkJoinPool.html. Acesso em: 30 out. 2026.

SEDGEWICK, Robert; WAYNE, Kevin. Elementary sorts. In: Algorithms, 4th edition: booksite. Princeton: Princeton University, [s. d.a]. Disponível em: https://algs4.cs.princeton.edu/21elementary/. Acesso em: 30 out. 2026.

SEDGEWICK, Robert; WAYNE, Kevin. Sorting. In: Algorithms, 4th edition: booksite. Princeton: Princeton University, [s. d.b]. Disponível em: https://algs4.cs.princeton.edu/20sorting/. Acesso em: 30 out. 2026.

SEDGEWICK, Robert; WAYNE, Kevin. Quicksort. In: Algorithms, 4th edition: booksite. Princeton: Princeton University, [s. d.c]. Disponível em: https://algs4.cs.princeton.edu/23quicksort/index.php. Acesso em: 30 out. 2026.

Anexos

Repositório GitHub: https://github.com/VitorVVC/av2_algoritmo. O código está em src/main/java/br/edu/unifor/cpc, com Sorts.java, Benchmark.java e Main.java. Os resultados originais e o resumo estão em results/completo.csv e results/resumo.csv; as figuras e a análise estão em docs. Para validar e reproduzir, executar sh run.sh --test e sh run.sh --full na raiz do projeto. A análise e as figuras podem ser regeneradas por scripts/analisar.py com Python e matplotlib.